AI Transit Pipeline

Complete Installation Guide

COMPONENTS

Pipeline	`ai_transit.sh` · `fetch_repo.sh` · `scan_pipeline.sh`
Reporting	`generate_excel_report.py` · `selfcheck.py`
Air-gap	`prepare_offline_install.sh` · `prepare_offline_cache.sh` · `verify_offline_install.sh`
Container	`docker-run.sh` · `Dockerfile`

Version 3.2 · 2026-10-22

6-layer security gateway for AI-generated repositories

Contents

The shape of it . . . 6
Stage A — Install everything (machine connected) . . . 6
Stage B — Make each tool work offline (machine still connected) . . . 8
Stage C — Disconnect and verify . . . 9
C1 — Scan something real . . . 9
C2 — Keep it working . . . 10
6.1 betterleaks — Secret Detection (Layer 1) . . . 11
6.2 detect-secrets — Entropy Scanning (Layer 1) . . . 12
6.3 ClamAV — Antivirus (Layer 1) . . . 12
6.4 YARA — Custom IOC Rules (Layer 1) . . . 14
6.5 Semgrep — OWASP / CWE / CERT Static Analysis (Layer 2) . . . 14
6.6 trivy — SCA / CVE (Layer 3) . . . 15
6.7 pip-audit — Python CVE (Layer 3) . . . 17
6.8 safety — Python Advisories (Layer 3) . . . 18
6.9 npm + npm audit — Node.js CVE (Layer 3) . . . 18
6.10 Bandit — Python SAST (Layer 5) . . . 19
6.11 ShellCheck — Shell Script SAST (Layer 5) . . . 20
6.12 cppcheck — C/C++ SAST (Layer 5) . . . 21
6.13 hadolint — Dockerfile Linter (Layer 5) . . . 22
6.14 checkov — Terraform / IaC SAST (Layer 5) . . . 23
6.15 ScanCode Toolkit — Licence & Copyright (Layer 6) . . . 24
7.1 Clone from GitHub . . . 25
7.2 Install Python dependencies for report generation . . . 25
7.3 Make scripts executable . . . 25
7.4 Verify scripts . . . 25
7.5 Run the test suite . . . 26
7.6 Generate the integrity manifest . . . 26
9.1 Command-line flags . . . 26
9.2 Environment variables . . . 26
9.3 Per-repo exclusions . . . 27
10.1 Final step — verify the install works offline . . . 29
11.1 Why --offline is required, and not merely advisable . . . 29
11.2 Per-tool reference . . . 30
11.2.1 Build files — Makefile, .mk, .mak . . . 31
11.2.2 HTML — .html, .htm, .xhtml . . . 32
11.2.3 PDF scanning — no staging, and why that matters . . . 33

11.2.4 Traversal and key material — the rules that run on every file . . . 34
11.3 Environment variables . . . 34
11.4 Building the cache — on a connected host . . . 35
11.5 Transferring and verifying . . . 35
11.6 Running . . . 35
11.7 Confirming a clean offline run . . . 35
11.8 Coverage — proving the scan actually ran . . . 36
11.9 Keeping the cache current . . . 36
12.1 Choose a path . . . 37
12.2 Path A — Docker image transfer . . . 37
12.3 Path B — Native package staging . . . 38
12.4 Verify the installation . . . 39
12.5 Then stage the scan data . . . 39
13.1 Quick smoke test (local directory) . . . 40
13.2 FAIL test — embedded secret . . . 40
13.3 FAIL test — SQL injection (OWASP A03) . . . 41
13.4 FAIL test — vulnerable dependency (CVE) . . . 41
13.5 WARN test — risky licence (GPL) . . . 41
13.6 FAIL test — Dockerfile misconfigurations . . . 42
13.7 Scan a real public GitHub repository . . . 42
13.8 Run the self-check to verify the pipeline itself . . . 42
13.9 CI mode — quiet verdict with severity filter . . . 43
13.10 Diff mode — scan only changed files (PR workflow) . . . 43
13.11 Private repository scan . . . 43
13.12 Report-only mode (audit without blocking) . . . 43
13.13 Air-gapped scan . . . 43
14.1 Output formats and selective checks . . . 44
14.2 The bundle manifest — generate it at install time . . . 44
Dedicated service account . . . 44
Network isolation (allow only github.com outbound) . . . 44
Quarantine filesystem (noexec) . . . 44
Weekly tool updates (cron) . . . 45
Log rotation . . . 45
17.1 What each layer covers . . . 47
17.2 Fixtures . . . 47
17.3 Adding a rule . . . 47
19.1 Build arguments . . . 48
19.2 Why the digests matter . . . 48

19.3 Running through the wrapper 49

1. Overview

LAYER	DETECTS	TOOLS	FAILS ON
L1	**Secrets & malware**	betterleaks · detect-secrets · ClamAV · YARA	**credential or malware found**
L2	**OWASP Top 10 · CWE Top 25**	Semgrep (4 rulesets, one pass)	**ERROR / WARNING finding**
L3	**Dependency CVEs**	trivy · pip-audit · safety · npm audit	**HIGH / CRITICAL CVE**
L4	**Built-in patterns**	grep rules — always available, no tool needed	**CWE-798 / 22 / 918 / 327 / 338**
L5	**Per-language SAST**	Bandit · ShellCheck · cppcheck · hadolint · checkov	**tool-specific finding**
L6	**Licence & package CVE**	ScanCode Toolkit	**risky licence → WARN · CVE → FAIL**

Any layer may FAIL. One FAIL quarantines the repository — the verdict is PASS only when none do.

Figure 1 — the six scanning layers, the tools in each, and what makes each one fail.

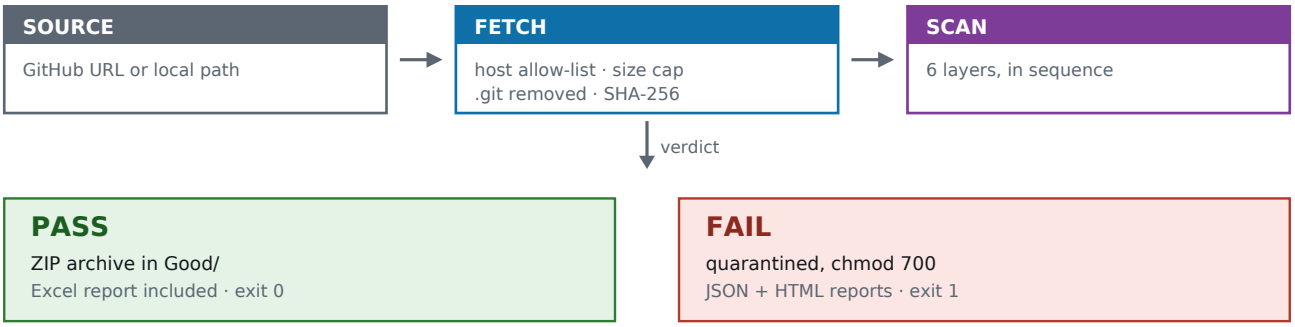


Figure 2 — end-to-end flow from source to verdict.

The AI Transit Pipeline is a 6-layer security gateway that scans AI-generated code before enterprise import.

Layer	Tools	Blocks on
L1 — Secrets & AV	betterleaks, detect-secrets, ClamAV, YARA	Secret/credential/malware detected
L2 — OWASP/CWE	Semgrep	ERROR/WARNING severity finding
L3 — SCA/CVE	trivy, pip-audit, safety, npm audit	HIGH/CRITICAL CVE in dependency
L4 — Patterns	grep (built-in)	CWE-798/22/918/327/338 match

Layer	Tools	Blocks on
L5 — Per-type SAST	Bandit, ShellCheck, cppcheck, hadolint, checkov, Semgrep per-lang	Tool-specific finding (also covers Rust, Kotlin, C#)
L6 — Licence	ScanCode Toolkit	CRITICAL/HIGH CVE in detected package

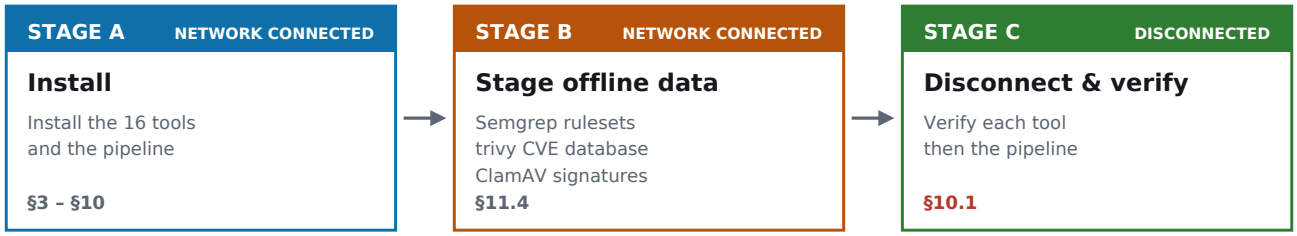
Verdict: any FAIL in any layer → package quarantined. WARNs are logged but do not block.

2. End-to-End Walkthrough — Blank Machine to Verified Offline Install

This is the spine of the manual. It takes one machine from bare OS to a fully offline-capable installation that has been *proved* to work disconnected.

Later sections are the detail behind each step; follow this one and refer to them as directed.

The shape of it



Stage C is not optional.
Until it has run, “offline-ready” is an assumption — and the failure it guards against is silent.

Figure 3 — the three stages of the walkthrough, and the network state each runs in.

```
┌─ Stage A ──────────┐   ┌─ Stage B ──────────┐   ┌─ Stage C ──────────┐
│  CONNECTED         │   │  CONNECTED         │   │  DISCONNECTED      │
│                    │   │                    │   │                    │
│  Install the 16    │──▶│  Stage the data    │──▶│  Pull the cable    │
│  tools + pipeline  │   │  each tool reads   │   │  Verify per tool   │
│                    │   │  offline           │   │  Verify pipeline   │
└────────────────────┘   └────────────────────┘   └────────────────────┘
```

The machine is connected for A and B, then disconnected for C. **Stage C is not optional.** Until it has run, "offline-ready" is an assumption, and the failure mode it protects against — a scan that reports PASS having examined nothing — is silent.

Building on one machine and deploying to another? Do Stages A and B here, then §12 covers packaging and transferring the result. Stage C is run on the target.

Stage A — Install everything (machine connected)

Step	Action	Section
A1	Confirm the OS, hardware and bash version	§3
A2	Install the system packages	§4
A3	Create the Python virtual environment	§5
A4	Install the 16 tools, one at a time	§6
A5	Install the pipeline scripts	§7
A6	Create the working directories	§8
A7	Run the full verification	§10

Condensed, for a fresh Ubuntu 22.04 / 24.04 host:

```
# A2 — system packages
sudo apt-get update && sudo apt-get install -y \
    bash git curl ca-certificates wget gpg rsync \
    jq zip unzip file coreutils \
    python3 python3-pip python3-venv build-essential golang-go \
    nodejs npm shellcheck cppcheck clamav clamav-daemon clamav-freshclam yara

# A3 — virtual environment
python3 -m venv /opt/ai-transit/venv
source /opt/ai-transit/venv/bin/activate

# A4 — Python tools
pip install --upgrade pip
pip install openpyxl reportlab python-docx detect-secrets bandit \
            pip-audit safety semgrep checkov scancode-toolkit

# A4 — Go tool
go install github.com/betterleaks/betterleaks@latest
sudo cp ~/go/bin/betterleaks /usr/local/bin/

# A4 — standalone binaries (pin versions; see §19.1 for digests)
curl -sfL "https://github.com/aquasecurity/trivy/releases/download/v0.58.2/trivy_0.58.2_Linux-64bit.tar.gz" \
    | sudo tar -xzf - -C /usr/local/bin trivy
sudo curl -sSL "https://github.com/hadolint/hadolint/releases/download/v2.12.0/hadolint-Linux-x86_64" \
    -o /usr/local/bin/hadolint && sudo chmod +x /usr/local/bin/hadolint

# A4 — ClamAV signatures (needed now; staged again in Stage B)
sudo systemctl stop clamav-freshclam 2>/dev/null || true
sudo freshclam
sudo systemctl start clamav-freshclam 2>/dev/null || true

# A5/A6 — pipeline and directories
sudo mkdir -p /opt/ai-transit/{fetch,quarantine,approved,reports,logs,yara-rules}
sudo chmod 700 /opt/ai-transit/quarantine
```

A7 — confirm every tool is actually present before going further:

```
for t in betterleaks detect-secrets clamscan yara semgrep trivy \
         bandit shellcheck cppcheck hadolint checkov scancode; do
    command -v "$t" >/dev/null && echo "  present : $t" || echo "  MISSING : $t"
done
./tests/run_tests.sh          # Expected: ✔ 108/108 passed
```

A tool missing here will still be missing offline, where it is much harder to notice. Resolve every `MISSING` before continuing.

Stage B — Make each tool work offline (machine still connected)

Eight of the tools already work offline; five need data staged first; four cannot work offline at all. This is the table that matters:

Tool	Layer	Offline requirement	How it is satisfied
betterleaks	L1	None — rules compiled in	—
detect-secrets	L1	None — rules embedded	—
ClamAV	L1	**Signature database**	`freshclam`, then signatures copied to the cache
YARA	L1	Your own `.yar` files	Copy them to `$WORK_DIR/yara-rules/`
Semgrep	L2	**Ruleset YAML per ruleset**	`semgrep --config p/<name> --dump-config`
trivy	L3	**Vulnerability database**	`trivy fs --download-db-only --cache-dir`
pip-audit	L3	*Impossible* — remote service	Covered by trivy instead
safety	L3	*Impossible* — remote service	Covered by trivy instead
npm audit	L3	*Impossible* — remote registry	Covered by trivy instead
grep rules	L4	None — built in	—
Bandit	L5	None — local AST analysis	—
ShellCheck	L5	None	—
cppcheck	L5	None	—
hadolint	L5	None	—
checkov	L5	None, but must not fetch schemas	`--skip-download`, applied automatically
ScanCode	L6	None for licence/copyright	`--vulnerability` dropped automatically

One command stages everything in the middle column:

```
./prepare_offline_cache.sh /opt/ai-transit/offline-cache
```

Read its summary. It reports what it staged and what it could not, and a skipped item here becomes a silent gap later.

Verify the staged data is plausible before disconnecting:

```
cd /opt/ai-transit/offline-cache
ls -1 semgrep-rules/     # expect 5 .yaml files
du -sh trivy-db/         # expect several hundred MB — a few KB means it failed
ls -1 clamav/            # expect *.cvd or *.cld
date -u +%Y-%m-%d > .cache_built_on
```

Make offline the default, so a later operator who forgets the flag still gets offline behaviour instead of a scan that hangs on network timeouts:

```
cat >> ~/.bashrc <<'EOF'
export WORK_DIR=/opt/ai-transit
export OFFLINE=true
export OFFLINE_CACHE=/opt/ai-transit/offline-cache
EOF
source ~/.bashrc
```

Stage C — Disconnect and verify

Disconnect the machine. Physically unplug it, or:

```
sudo ip link set "$(ip route | awk '/default/{print $5; exit}')" down
```

Then run the verification, which checks each tool individually before running the pipeline:

```
./verify_offline_install.sh
```

Per-tool checking matters because the pipeline degrades quietly: a tool that cannot work offline produces an empty result, and an empty result is indistinguishable from a clean one. Checking tools one at a time turns that silence into a named failure.

Expected output on a correctly prepared machine:

```
── Network state
  ✔ Network is unreachable — this is a genuine offline test.

── Group A — no staged data required
  ✔ betterleaks works offline
  ✔ detect-secrets works offline
  ✔ bandit works offline
  ...

── Group B — requires staged data
  ✔ semgrep runs from staged rules (5 ruleset file(s))
  ✔ semgrep cannot reach its registry (expected offline)
  ✔ trivy scans from the staged database
  ✔ clamscan runs against signatures in /opt/ai-transit/offline-cache/clamav
  ...

── End-to-end — pipeline scan with coverage check
  ✔ offline pipeline run completed
      [OK ] L2_owasp_cwe               ran
      [OK ] L3_dependency_cve          ran
      ...
  ✔ all required layers ran offline

  ✔  13/13 checks passed
```

Exit status `0` means the installation is offline-ready. Any failure names the tool and points at the staging step that fixes it.

Cannot disconnect the machine? `--simulate` forces outbound HTTP to a dead port, which catches the common case:

```
./verify_offline_install.sh --simulate
```

It is weaker than a real disconnection — it does not defeat cached DNS or a local proxy — so treat a pass as provisional until you have tested genuinely disconnected.

C1 — Scan something real

```
./ai_transit.sh --offline /path/to/some/repo

REPORT=$(ls -t "$WORK_DIR"/reports/report_*.json | head -1)
```

```
python3 -c "
import json,sys
d=json.load(open(sys.argv[1]))
print('verdict :', d['verdict'])
print('gaps    :', ', '.join(d['coverage_gaps']) or 'none')
" "$REPORT"
```

`L1_ioc_yara` is a legitimate gap if you have written no custom IOC rules. Everything else should be absent. §11.8 explains the coverage block; the step-by-step operating procedure is **OFFLINE_RUNBOOK.md**.

C2 — Keep it working

The install is static, but the staged data is not. A stale CVE database reports clean for everything published since it was built — the same outcome as not scanning, and harder to notice.

Reconnect, re-run `prepare_offline_cache.sh`, disconnect, re-run `verify_offline_install.sh`. **Weekly** for the trivy database and ClamAV signatures; monthly is sufficient for Semgrep rulesets. See §11.9.

3. Prerequisites

Supported platforms: Ubuntu 22.04 LTS / Debian 12 (production), Ubuntu 24.04 LTS (recommended). Windows users: install via **WSL2** (see Section 4 of base guide).

Minimum hardware: 4-core CPU, 8 GB RAM, 20 GB free disk.

4. Core System Packages

Install the baseline packages first. These are required for the pipeline to run at all.

```
sudo apt-get update && sudo apt-get upgrade -y

sudo apt-get install -y \
    bash git curl ca-certificates wget gpg \
    jq zip unzip file coreutils \
    python3 python3-pip python3-venv \
    build-essential golang-go \
    nodejs npm
```

Verify:

```
bash   --version | head -1   # expect: GNU bash, version 5.x
git    --version             # expect: git version 2.x
python3 --version            # expect: Python 3.10+
go     version               # expect: go1.21+
node   --version             # expect: v18+ or v20+
jq     --version             # expect: jq-1.6+
```

5. Python Virtual Environment

Using a dedicated venv avoids conflicts with system packages and makes the installation self-contained.

```
python3 -m venv /opt/ai-transit/venv
echo 'source /opt/ai-transit/venv/bin/activate' >> ~/.bashrc
source /opt/ai-transit/venv/bin/activate
pip install --upgrade pip
```

> All `pip install` commands in this guide assume the venv is active.

Verify:

```
which python3     # must show /opt/ai-transit/venv/bin/python3
python3 --version # Python 3.10+
```

6. Tool-by-Tool Installation

Each section covers: finding the latest version, installing, verifying, offline setup, and a functional test.

6.1 betterleaks — Secret Detection (Layer 1)

What it does: scans all file types for secrets, API keys, credentials and tokens. Successor to gitleaks with a more expressive allowlist system.

Find the latest version

```
# Query GitHub API for latest release tag
curl -s https://api.github.com/repos/betterleaks/betterleaks/releases/latest \
    | jq -r '.tag_name'
```

Install (Go — recommended, produces a static binary)

```
go install github.com/betterleaks/betterleaks@latest
sudo cp ~/go/bin/betterleaks /usr/local/bin/betterleaks
```

Install (Homebrew — macOS / Linux with brew)

```
brew install betterleaks
```

Install (Fedora / RHEL)

```
sudo dnf install betterleaks
```

Verify installation

```
betterleaks --version
# Expected output: betterleaks version x.y.z
```

Offline operation

betterleaks has **no external database** — all detection rules are embedded in the binary. Once installed, it works fully offline.

Functional test

```
# Create a test file with a fake secret
mkdir /tmp/bl-test
echo 'AWS_SECRET_ACCESS_KEY="wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"' \
    > /tmp/bl-test/config.py

betterleaks dir /tmp/bl-test -v
# Expected: non-zero exit code, finding reported for config.py

# Clean file — should PASS
echo 'import os; key = os.environ["AWS_SECRET_ACCESS_KEY"]' \
    > /tmp/bl-test/config_clean.py
betterleaks dir /tmp/bl-test/config_clean.py -v
# Expected: exit code 0

rm -rf /tmp/bl-test
```

6.2 detect-secrets — Entropy Scanning (Layer 1)

What it does: complements betterleaks with Shannon entropy analysis and regex-based patterns to find high-entropy strings that look like secrets.

Find the latest version

```
pip index versions detect-secrets 2>/dev/null | head -1
# or:
curl -s https://pypi.org/pypi/detect-secrets/json | jq -r '.info.version'
```

Install

```
pip install detect-secrets
```

Verify installation

```
detect-secrets --version
# Expected: x.y.z
```

Offline operation

No external database. Works fully offline once installed.

Functional test

```
mkdir /tmp/ds-test
echo 'password = "Sup3rS3cr3tP@ssw0rd!"' > /tmp/ds-test/app.py

detect-secrets scan /tmp/ds-test/app.py
# Expected: JSON with non-empty "results" containing app.py

echo 'password = os.getenv("PASSWORD")' > /tmp/ds-test/app_clean.py
detect-secrets scan /tmp/ds-test/app_clean.py
# Expected: JSON with empty "results" {}

rm -rf /tmp/ds-test
```

6.3 ClamAV — Antivirus (Layer 1)

What it does: scans files against a database of known malware signatures. Requires regular DB updates to stay effective.

Find the latest version

```
apt-cache policy clamav | grep Candidate
# or for the latest upstream:
curl -s https://www.clamav.net/downloads | grep -oP 'clamav-\K[0-9.]+(?=\.tar)' | head -1
```

Install

```
sudo apt-get install -y clamav clamav-daemon clamav-freshclam
```

Update virus database (required before first use)

```
sudo systemctl stop clamav-freshclam
sudo freshclam
sudo systemctl start clamav-freshclam
sudo systemctl enable clamav-freshclam
```

Verify installation

```
clamscan --version
# Expected: ClamAV x.y.z/NNNNN/...
freshclam --version
# Expected: ClamAV x.y.z
```

Offline operation

ClamAV works offline once the virus DB is downloaded. To prepare for offline use:

```
# On a machine with internet access, download the DB files
sudo freshclam

# The DB files are stored at (copy these to the offline machine):
ls /var/lib/clamav/
# main.cvd (or main.cld), daily.cvd (or daily.cld), bytecode.cvd

# On the offline machine, copy the DB files to /var/lib/clamav/
# then reload:
sudo systemctl restart clamav-daemon
```

Functional test

```
# ClamAV includes the EICAR test signature — a safe test virus string
echo 'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*' \
    > /tmp/eicar.txt

clamscan /tmp/eicar.txt
# Expected: /tmp/eicar.txt: Eicar-Signature FOUND
# Expected exit code: 1

clamscan /bin/ls
# Expected: /bin/ls: OK
# Expected exit code: 0

rm /tmp/eicar.txt
```

6.4 YARA — Custom IOC Rules (Layer 1)

What it does: loads organisation-specific YARA rules from `yara-rules/` and matches them against every file. Fully customisable pattern engine.

Find the latest version

```
apt-cache policy yara | grep Candidate
# or from GitHub:
curl -s https://api.github.com/repos/VirusTotal/yara/releases/latest \
    | jq -r '.tag_name'
```

Install

```
sudo apt-get install -y yara
```

Verify installation

```
yara --version
# Expected: x.y.z
```

Offline operation

YARA has no external database — all rules are local `.yar` files. Works fully offline.

Create a minimal test rule

```
mkdir -p /opt/ai-transit/yara-rules

cat > /opt/ai-transit/yara-rules/test.yar << 'EOF'
rule Suspicious_Base64_Exec {
    meta:
        description = "Detects base64-encoded exec calls"
    strings:
        $b64_exec = /eval\(base64_decode\(/ nocase
    condition:
        $b64_exec
}
EOF
```

Functional test

```
echo '<?php eval(base64_decode("dW5saW5r..."); ?>' > /tmp/test.php

yara /opt/ai-transit/yara-rules/test.yar /tmp/test.php
# Expected: Suspicious_Base64_Exec /tmp/test.php

echo '<?php echo "Hello World"; ?>' > /tmp/clean.php
yara /opt/ai-transit/yara-rules/test.yar /tmp/clean.php
# Expected: (no output — no match)

rm /tmp/test.php /tmp/clean.php
```

6.5 Semgrep — OWASP / CWE / CERT Static Analysis (Layer 2)

What it does: runs four security rulesets (OWASP Top 10, CWE Top 25, security-audit, secrets) across all supported languages using pattern matching on the AST.

Find the latest version

```
curl -s https://pypi.org/pypi/semgrep/json | jq -r '.info.version'
```

Install

```
pip install semgrep
```

Verify installation

```
semgrep --version
# Expected: 1.x.x
```

Offline operation

Semgrep needs internet access **only on the first run** to download rules. Prepare for offline:

```
# On a machine with internet, pre-download rules to a local directory
mkdir -p /opt/ai-transit/semgrep-rules
semgrep --config p/owasp-top-ten   --dump-config > /opt/ai-transit/semgrep-rules/owasp.yaml
semgrep --config p/cwe-top-25      --dump-config > /opt/ai-transit/semgrep-rules/cwe.yaml
semgrep --config p/security-audit  --dump-config > /opt/ai-transit/semgrep-rules/audit.yaml
semgrep --config p/secrets         --dump-config > /opt/ai-transit/semgrep-rules/secrets.yaml

# On the offline machine, run against local rules:
semgrep --config /opt/ai-transit/semgrep-rules/ <repo_dir>
```

Functional test

```
cat > /tmp/test_sqli.py << 'EOF'
import sqlite3
def get_user(username):
    conn = sqlite3.connect("db.sqlite3")
    query = "SELECT * FROM users WHERE name = '" + username + "'"
    return conn.execute(query).fetchall()
EOF

semgrep --config p/owasp-top-ten --json /tmp/test_sqli.py \
    | jq '.results | length'
# Expected: 1 or more (SQL injection detected)

cat > /tmp/test_clean.py << 'EOF'
import sqlite3
def get_user(username):
    conn = sqlite3.connect("db.sqlite3")
    return conn.execute("SELECT * FROM users WHERE name = ?", (username,)).fetchall()
EOF

semgrep --config p/owasp-top-ten --json /tmp/test_clean.py \
    | jq '.results | length'
# Expected: 0

rm /tmp/test_sqli.py /tmp/test_clean.py
```

6.6 trivy — SCA / CVE (Layer 3)

What it does: scans dependency manifests (requirements.txt, package-lock.json, go.sum, pom.xml, Cargo.lock …) against NVD, OSV and GitHub Advisory databases.

Find the latest version

```
curl -s https://api.github.com/repos/aquasecurity/trivy/releases/latest \
    | jq -r '.tag_name'
```

Install (official script — recommended)

```
curl -sfL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh \
    | sudo sh -s -- -b /usr/local/bin
```

Install (apt repository)

```
sudo apt-get install -y wget gnupg
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key \
    | sudo gpg --dearmor -o /usr/share/keyrings/trivy.gpg
echo "deb [signed-by=/usr/share/keyrings/trivy.gpg] \
    https://aquasecurity.github.io/trivy-repo/deb generic main" \
    | sudo tee /etc/apt/sources.list.d/trivy.list
sudo apt-get update && sudo apt-get install -y trivy
```

Verify installation

```
trivy --version
# Expected: Version: x.y.z
```

Download vulnerability database (required before first offline use)

```
# Pre-download the CVE database (requires internet)
trivy image --download-db-only
trivy image --download-java-db-only

# DB is cached at:
ls ~/.cache/trivy/db/
```

Offline operation

```
# After downloading the DB, run offline:
trivy fs <repo_dir> \
    --scanners vuln \
    --severity HIGH,CRITICAL \
    --skip-db-update \
    --offline-scan
```

Functional test

```
mkdir /tmp/trivy-test

# Create a requirements.txt with a known vulnerable package
cat > /tmp/trivy-test/requirements.txt << 'EOF'
Pillow==9.0.0
requests==2.18.0
EOF

trivy fs /tmp/trivy-test \
    --scanners vuln \
    --severity HIGH,CRITICAL \
    --format table
# Expected: findings for known CVEs in Pillow 9.0.0 and requests 2.18.0
```

```
# Clean requirements
cat > /tmp/trivy-test/requirements_clean.txt << 'EOF'
Pillow==10.3.0
requests==2.32.3
EOF

trivy fs /tmp/trivy-test/requirements_clean.txt \
    --scanners vuln \
    --severity HIGH,CRITICAL \
    --format table
# Expected: No vulnerabilities found

rm -rf /tmp/trivy-test
```

6.7 pip-audit — Python CVE (Layer 3)

What it does: audits Python requirements files against the Python Packaging Advisory Database (PyPA) and OSV.

Find the latest version

```
curl -s https://pypi.org/pypi/pip-audit/json | jq -r '.info.version'
```

Install

```
pip install pip-audit
```

Verify installation

```
pip-audit --version
# Expected: pip-audit x.y.z
```

Offline operation

pip-audit requires internet to query the PyPA/OSV database. For offline use, use trivy with `--offline-scan` instead (covers the same packages).

Functional test

```
cat > /tmp/req_vuln.txt << 'EOF'
Pillow==9.0.0
requests==2.18.0
EOF

pip-audit -r /tmp/req_vuln.txt
# Expected: one or more vulnerabilities listed, exit code 1

cat > /tmp/req_clean.txt << 'EOF'
Pillow==10.3.0
requests==2.32.3
EOF

pip-audit -r /tmp/req_clean.txt
# Expected: No known vulnerabilities found, exit code 0

rm /tmp/req_vuln.txt /tmp/req_clean.txt
```

6.8 safety — Python Advisories (Layer 3)

What it does: checks Python dependencies against the Safety DB (PyUp.io), a curated advisory database with additional entries not always in OSV.

Find the latest version

```
curl -s https://pypi.org/pypi/safety/json | jq -r '.info.version'
```

Install

```
pip install safety
```

Verify installation

```
safety --version
# Expected: safety, version x.y.z
```

Offline operation

safety requires internet for DB queries. For offline environments, use trivy as the primary SCA scanner.

Functional test

```
cat > /tmp/req_vuln.txt << 'EOF'
Pillow==9.0.0
EOF

safety check -r /tmp/req_vuln.txt
# Expected: vulnerabilities found, exit code 64

cat > /tmp/req_clean.txt << 'EOF'
Pillow==10.3.0
EOF

safety check -r /tmp/req_clean.txt
# Expected: No known security vulnerabilities found, exit code 0

rm /tmp/req_vuln.txt /tmp/req_clean.txt
```

6.9 npm + npm audit — Node.js CVE (Layer 3)

What it does: built into npm — audits `package-lock.json` against the npm security advisory registry for known CVEs in Node.js dependencies.

Find the latest version

```
npm --version
node --version
# npm is bundled with Node.js; update both together
```

Install (via NodeSource — LTS)

```
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Verify installation

```
node --version    # Expected: v20.x.x or v22.x.x
```

```
npm  --version    # Expected: 10.x.x or later
npm audit --version 2>/dev/null || echo "npm audit is built-in"
```

Offline operation

npm audit requires internet to query the npm advisory registry. No offline mode is available.

Functional test

```
mkdir /tmp/npm-test && cd /tmp/npm-test

# Create a package.json with a known vulnerable package
cat > package.json << 'EOF'
{
  "name": "test",
  "version": "1.0.0",
  "dependencies": {
    "lodash": "4.17.4"
  }
}
EOF

npm install --package-lock-only 2>/dev/null
npm audit --json | jq '.metadata.vulnerabilities'
# Expected: object with non-zero high/critical counts

cd / && rm -rf /tmp/npm-test
```

6.10 Bandit — Python SAST (Layer 5)

What it does: Python-specific static analyser with 100+ plugins detecting SQL injection, shell injection, hardcoded passwords, weak crypto, insecure deserialization, and more.

Find the latest version

```
curl -s https://pypi.org/pypi/bandit/json | jq -r '.info.version'
```

Install

```
pip install bandit
```

Verify installation

```
bandit --version
# Expected: bandit x.y.z  (Python x.y.z)
```

Offline operation

Bandit has no external database. Works fully offline.

Functional test

```
cat > /tmp/test_bandit.py << 'EOF'
import subprocess
import hashlib

# B602 - subprocess with shell=True
def run(cmd):
    subprocess.call(cmd, shell=True)
```

```
# B324 - use of weak hash
def hash_password(pwd):
    return hashlib.md5(pwd.encode()).hexdigest()
EOF

bandit -ll /tmp/test_bandit.py
# Expected: Issue: [B602] ... [B324] ... exit code 1

cat > /tmp/test_bandit_clean.py << 'EOF'
import subprocess
import hashlib

def run(cmd_list):
    subprocess.call(cmd_list)

def hash_password(pwd):
    return hashlib.sha256(pwd.encode()).hexdigest()
EOF

bandit -ll /tmp/test_bandit_clean.py
# Expected: No issues identified, exit code 0

rm /tmp/test_bandit.py /tmp/test_bandit_clean.py
```

6.11 ShellCheck — Shell Script SAST (Layer 5)

What it does: static analyser for bash/sh/dash/ksh. Detects quoting errors, command injection risks, undefined variables, deprecated syntax, and unsafe patterns.

Find the latest version

```
apt-cache policy shellcheck | grep Candidate
# or latest binary from GitHub:
curl -s https://api.github.com/repos/koalaman/shellcheck/releases/latest \
    | jq -r '.tag_name'
```

Install (apt)

```
sudo apt-get install -y shellcheck
```

Install (latest binary from GitHub)

```
VERSION=$(curl -s https://api.github.com/repos/koalaman/shellcheck/releases/latest \
    | jq -r '.tag_name')
curl -sSL "https://github.com/koalaman/shellcheck/releases/download/${VERSION}/shellcheck-${VERSION}.linux.x86_64.tar.xz" \
    | tar -xJ --strip-components=1 -C /tmp
sudo mv /tmp/shellcheck /usr/local/bin/shellcheck
```

Verify installation

```
shellcheck --version
# Expected: version: x.y.z
```

Offline operation

ShellCheck has no external database. Works fully offline.

Functional test

```
cat > /tmp/test.sh << 'EOF'
#!/bin/bash
filename=$1
if [ $filename == "admin" ]; then
  echo "Hello $filename"
fi
rm -rf /$filename
EOF

shellcheck --severity=warning /tmp/test.sh
# Expected: SC2086 (unquoted variable), SC2115 (unsafe rm), exit code 1

cat > /tmp/test_clean.sh << 'EOF'
#!/bin/bash
filename="$1"
if [[ "$filename" == "admin" ]]; then
  echo "Hello $filename"
fi
EOF

shellcheck --severity=warning /tmp/test_clean.sh
# Expected: exit code 0

rm /tmp/test.sh /tmp/test_clean.sh
```

6.12 cppcheck — C/C++ SAST (Layer 5)

What it does: static analyser for C and C++ detecting buffer overflows, memory leaks, null-pointer dereferences, use-after-free, and undefined behaviour without compiling.

Find the latest version

```
apt-cache policy cppcheck | grep Candidate
# or from GitHub:
curl -s https://api.github.com/repos/danmar/cppcheck/releases/latest \
    | jq -r '.tag_name'
```

Install (apt)

```
sudo apt-get install -y cppcheck
```

Verify installation

```
cppcheck --version
# Expected: Cppcheck x.y
```

Offline operation

cppcheck has no external database. Works fully offline.

Functional test

```
cat > /tmp/test.cpp << 'EOF'
#include <string.h>
void test() {
    char buf[10];
    strcpy(buf, "Hello, this string is too long and will overflow the buffer!");
    int* p = new int(42);
```

```
    delete p;
    *p = 100;  // use after free
}
EOF

cppcheck --enable=warning,security --error-exitcode=1 /tmp/test.cpp
# Expected: errors for buffer overflow and use-after-free, exit code 1

cat > /tmp/test_clean.cpp << 'EOF'
#include <cstring>
void test() {
    char buf[100];
    strncpy(buf, "Hello", sizeof(buf) - 1);
    buf[sizeof(buf) - 1] = '\0';
}
EOF

cppcheck --enable=warning,security --error-exitcode=1 /tmp/test_clean.cpp
# Expected: no errors, exit code 0

rm /tmp/test.cpp /tmp/test_clean.cpp
```

6.13 hadolint — Dockerfile Linter (Layer 5)

What it does: enforces Dockerfile best practices and detects security misconfigurations: running as root, `:latest` tags, baked-in secrets, ADD vs COPY, shell injection.

Find the latest version

```
curl -s https://api.github.com/repos/hadolint/hadolint/releases/latest \
    | jq -r '.tag_name'
```

Install (binary)

```
VERSION=$(curl -s https://api.github.com/repos/hadolint/hadolint/releases/latest \
    | jq -r '.tag_name')
curl -sSL "https://github.com/hadolint/hadolint/releases/download/${VERSION}/hadolint-Linux-x86_64" \
    -o /usr/local/bin/hadolint
sudo chmod +x /usr/local/bin/hadolint
```

Verify installation

```
hadolint --version
# Expected: Haskell Dockerfile Linter x.y.z
```

Offline operation

hadolint has no external database. Works fully offline.

Functional test

```
cat > /tmp/Dockerfile_bad << 'EOF'
FROM ubuntu:latest
RUN apt-get update && apt-get install -y curl
ADD https://example.com/script.sh /tmp/
ENV PASSWORD=mysecretpassword
RUN /tmp/script.sh
EOF
```

```
hadolint /tmp/Dockerfile_bad
# Expected: DL3007 (latest tag), DL3009 (delete apt cache), SC2046, etc.

cat > /tmp/Dockerfile_good << 'EOF'
FROM ubuntu:22.04
RUN apt-get update \
    && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
COPY script.sh /tmp/script.sh
USER nobody
EOF

hadolint /tmp/Dockerfile_good
# Expected: no output or only style suggestions, exit code 0

rm /tmp/Dockerfile_bad /tmp/Dockerfile_good
```

6.14 checkov — Terraform / IaC SAST (Layer 5)

What it does: static analysis for Terraform, CloudFormation, Kubernetes YAML, Ansible and ARM templates. Maps findings to CIS Benchmarks, NIST, SOC2, and OWASP. 1000+ built-in checks.

Find the latest version

```
curl -s https://pypi.org/pypi/checkov/json | jq -r '.info.version'
```

Install

```
pip install checkov
```

Verify installation

```
checkov --version
# Expected: x.y.z
```

Offline operation

checkov uses local checks only — works fully offline. No external DB required.

Functional test

```
mkdir /tmp/tf-test

cat > /tmp/tf-test/main.tf << 'EOF'
resource "aws_s3_bucket" "public_bucket" {
  bucket = "my-public-bucket"
  acl    = "public-read"
}

resource "aws_security_group" "open" {
  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

```
EOF

checkov -d /tmp/tf-test --framework terraform --compact
# Expected: FAILED checks (public S3 bucket, open SSH to world)

cat > /tmp/tf-test/main_secure.tf << 'EOF'
resource "aws_s3_bucket" "private_bucket" {
  bucket = "my-private-bucket"
}
resource "aws_s3_bucket_acl" "private" {
  bucket = aws_s3_bucket.private_bucket.id
  acl    = "private"
}
EOF

checkov -f /tmp/tf-test/main_secure.tf --framework terraform --compact
# Expected: all checks passed or significantly fewer failures

rm -rf /tmp/tf-test
```

6.15 ScanCode Toolkit — Licence & Copyright (Layer 6)

What it does: full-text licence detection using 30 000+ licence texts (SPDX), copyright notice extraction, package manifest detection, and CVE lookup in detected packages. Produces JSON/SPDX/CycloneDX reports.

Find the latest version

```
curl -s https://pypi.org/pypi/scancode-toolkit/json | jq -r '.info.version'
```

Install

```
pip install scancode-toolkit
```

> ScanCode is a large package (~500 MB installed). Installation may take 5–10 minutes.

Verify installation

```
scancode --version
# Expected: ScanCode version x.y.z
```

Offline operation

ScanCode works **fully offline** — all licence texts and detection logic are bundled in the package. No external DB or internet required at scan time.

Functional test

```
mkdir /tmp/sc-test

# File with a GPL licence header
cat > /tmp/sc-test/app.py << 'EOF'
# This program is free software: you can redistribute it and/or modify
# it under the terms of the GNU General Public License as published by
# the Free Software Foundation, either version 3 of the License, or
# (at your option) any later version.
# Copyright (C) 2024 Example Corp.
```

```
import os

def main():
    print("hello")
EOF

# File with a permissive MIT licence
cat > /tmp/sc-test/lib.py << 'EOF'
# MIT License
# Copyright (c) 2024 Example Corp
# Permission is hereby granted, free of charge, to any person obtaining a copy...
def helper():
    pass
EOF

scancode --license --copyright \
    --json-pp /tmp/sc-report.json \
    --quiet \
    /tmp/sc-test

# Check detections
jq '.files[] | {path: .path, licenses: [.license_detections[].matches[].spdx_license_expression]}' \
    /tmp/sc-report.json

# Expected: GPL-3.0-or-later detected in app.py, MIT in lib.py

rm -rf /tmp/sc-test /tmp/sc-report.json
```

7. Pipeline Scripts Installation

7.1 Clone from GitHub

```
git clone https://github.com/gaillotte/claude.git
cd claude
git checkout claude/vigilant-carson-f8twy0
```

7.2 Install Python dependencies for report generation

```
pip install openpyxl reportlab
```

7.3 Make scripts executable

```
chmod +x ai_transit.sh fetch_repo.sh scan_pipeline.sh
```

7.4 Verify scripts

```
bash -n ai_transit.sh    && echo "ai_transit.sh: syntax OK"
bash -n fetch_repo.sh    && echo "fetch_repo.sh: syntax OK"
bash -n scan_pipeline.sh && echo "scan_pipeline.sh: syntax OK"
python3 -c "import ast; ast.parse(open('generate_excel_report.py').read())" \
    && echo "generate_excel_report.py: syntax OK"
python3 -c "import ast; ast.parse(open('selfcheck.py').read())" \
```

```
    && echo "selfcheck.py: syntax OK"
```

7.5 Run the test suite

This is the fastest way to confirm the installation is sound. It needs no scanning tools — see §17.

```
./tests/run_tests.sh
# Expected: ✔ 108/108 passed
```

7.6 Generate the integrity manifest

```
python3 selfcheck.py --write-manifest
```

Do this once the bundle is in its final location. See §14.2 for why the manifest is generated at install time rather than shipped in version control.

8. Directory Structure Setup

```
sudo mkdir -p /opt/ai-transit/{fetch,quarantine,approved,reports,logs,yara-rules}
sudo chmod 700 /opt/ai-transit/quarantine
sudo chown -R $USER:$USER /opt/ai-transit
mkdir -p Good
```

Copy your YARA rules to `/opt/ai-transit/yara-rules/` (see §6.4 for a minimal test rule).

9. Environment Variables & Flags

9.1 Command-line flags

```
./ai_transit.sh [FLAGS] <repo_url_or_path> [branch]

  --quiet               Suppress all output except the final PASS/FAIL verdict (CI mode)
  --verbose             Full debug output including per-file scan details
  --min-severity LEVEL  Severity threshold: low | medium | high (default) | critical
                        Findings below the threshold are logged as WARN, not FAIL
  --since COMMIT        Diff mode: only scan files changed since this commit SHA
  --report-only         Always exit 0 even on FAIL (observation / audit mode).
                        Also leaves the fetched repo in place instead of
                        quarantining it, so nothing is moved or deleted.
  --no-zip              Skip creation of the approved ZIP archive
  --no-excel            Skip generation of the Excel report
  --offline             Air-gapped mode: no scanner attempts a network call.
                        Uses locally staged rules and databases, and reports any
                        layer that could not run. See section 10.
```

`--no-zip --no-excel` is the usual combination for CI, where the JSON and HTML reports are consumed by another job and the archive would only be discarded.

9.2 Environment variables

Variable	Default	Description
WORK_DIR	/opt/ai-transit	Root working directory
OUTPUT_DIR	<script_dir>/Good	Output directory for approved ZIPs
GITHUB_TOKEN	*(unset)*	Personal access token for private GitHub repos
MAX_SIZE_MB	500	Repository size limit in MB
MIN_SEVERITY	high	Minimum severity to block (`low\
VERBOSITY	normal	Log verbosity passed to scanner (`quiet\
SINCE_COMMIT	*(unset)*	Diff mode commit SHA (same as --since)
OFFLINE	false	Air-gapped mode (same as --offline)
OFFLINE_CACHE	$WORK_DIR/offline-cache	Root of staged offline assets
SEMGREP_RULES_DIR	$OFFLINE_CACHE/semgrep-rules	Exported Semgrep rulesets
TRIVY_CACHE_DIR	$OFFLINE_CACHE/trivy-db	trivy vulnerability database
CLAMAV_DB_DIR	$OFFLINE_CACHE/clamav	ClamAV signature files

Add persistent values to ~/.bashrc:

```
export WORK_DIR="/opt/ai-transit"
export OUTPUT_DIR="/opt/ai-transit/approved"
```

Or pass inline:

```
WORK_DIR=/data/ai-transit ./ai_transit.sh https://github.com/org/repo
```

9.3 Per-repo exclusions

Place either of these files in the **root of the scanned repository** (not the pipeline directory):

File	Format	Effect
.transitignore	gitignore-style patterns	Files matching patterns are excluded from all scan layers
.transit-allow.json	JSON array of {rule, path, reason}	Matching FAIL findings are downgraded to WARN

.transit-allow.json **example:**

```
[
  {
    "rule": "CWE-798",
    "path": "tests/fixtures/dummy_key.py",
    "reason": "Test fixture — not a real credential"
  },
  {
    "rule": "binary",
    "path": "assets/logo.png",
    "reason": "Approved binary asset"
  }
]
```

10. Full Installation Verification

Save this as `verify_install.sh` and run it after completing all sections above:

```
#!/usr/bin/env bash
# verify_install.sh — checks all required and optional tools

PASS=0; WARN=0; FAIL=0

check()   {
    local name="$1" cmd="$2"
    if command -v "$cmd" &>/dev/null; then
        echo -e "\033[32m[OK]\033[0m    $name"
        (( PASS++ ))
    else
        echo -e "\033[33m[WARN]\033[0m  $name — optional, not installed"
        (( WARN++ ))
    fi
}

require() {
    local name="$1" cmd="$2"
    if command -v "$cmd" &>/dev/null; then
        echo -e "\033[32m[OK]\033[0m    $name"
        (( PASS++ ))
    else
        echo -e "\033[31m[FAIL]\033[0m  $name — REQUIRED, not installed"
        (( FAIL++ ))
    fi
}

echo "=== Core system ==="
require "bash"        bash
require "git"         git
require "python3"     python3
require "pip"         pip
require "curl"        curl
require "jq"          jq
require "zip"         zip
require "sha256sum"   sha256sum
require "file"        file

echo ""
echo "=== Python report packages ==="
python3 -c "import openpyxl"  2>/dev/null \
    && { echo -e "\033[32m[OK]\033[0m    openpyxl"; (( PASS++ )); } \
    || { echo -e "\033[31m[FAIL]\033[0m  openpyxl — pip install openpyxl"; (( FAIL++ )); }
python3 -c "import reportlab" 2>/dev/null \
    && { echo -e "\033[32m[OK]\033[0m    reportlab"; (( PASS++ )); } \
    || { echo -e "\033[31m[FAIL]\033[0m  reportlab — pip install reportlab"; (( FAIL++ )); }

echo ""
echo "=== Layer 1 — Secrets & AV ==="
check "betterleaks"     betterleaks
check "detect-secrets"  detect-secrets
check "ClamAV"          clamscan
check "YARA"            yara

echo ""
echo "=== Layer 2 — OWASP/CWE ==="
check "Semgrep"         semgrep

echo ""
echo "=== Layer 3 — SCA/CVE ==="
check "trivy"           trivy
check "pip-audit"       pip-audit
```

```
check "safety"          safety
check "npm"             npm

echo ""
echo "=== Layer 5 — SAST ==="
check "Bandit"          bandit
check "ShellCheck"      shellcheck
check "cppcheck"        cppcheck
check "hadolint"        hadolint
check "checkov"         checkov

echo ""
echo "=== Layer 6 — Licence ==="
check "scancode"        scancode

echo ""
echo "=== Summary ==="
echo -e "  \033[32mOK: $PASS\033[0m  |  \033[33mOptional missing: $WARN\033[0m  |  \033[31mRequired missing: $FAIL\033[0m"
[[ $FAIL -eq 0 ]] \
    && echo -e "\033[32mInstallation COMPLETE — pipeline ready\033[0m" \
    || echo -e "\033[31mInstallation INCOMPLETE — fix FAIL items above\033[0m"
```

```
chmod +x verify_install.sh && ./verify_install.sh
```

10.1 Final step — verify the install works offline

Tool presence is not the same as tool *readiness*. Once the offline cache is staged (§11.4), disconnect the machine and run:

```
./verify_offline_install.sh
```

It checks each tool individually before running the pipeline, because the pipeline degrades quietly: a tool that cannot work offline yields an empty result, which looks exactly like a clean one. Full walkthrough: §2, Stage C.

11. Air-Gapped Operation

The pipeline supports fully disconnected operation through `--offline`. This section is the reference for what each tool needs in that mode.

11.1 Why `--offline` is required, and not merely advisable

Several scanners reach the network at **scan** time, not just at install time. Without `--offline` on an isolated host those calls are still attempted. They do not crash the run — each is individually guarded — but they block on DNS and TCP timeouts and then return nothing.

The consequence is worse than slowness. Layer 2 (OWASP/CWE) and Layer 3 (CVE) contribute no findings, while the report still presents them as having run. The verdict can be **PASS on a repository that was never meaningfully scanned**.

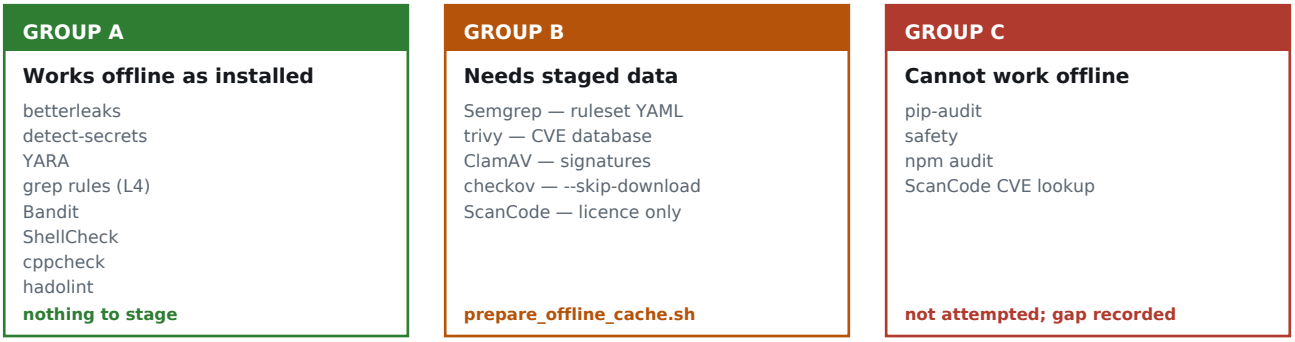
`--offline` changes that: each tool is pointed at locally staged data and given the flags that suppress update attempts and telemetry, and any layer that genuinely cannot run is recorded as an explicit `OFFLINE:` warning naming what did not execute. A silent gap becomes a stated one.

```
export OFFLINE_CACHE=/opt/ai-transit/offline-cache
./ai_transit.sh --offline /path/to/repo
```

> **For the step-by-step operating procedure** — building and transferring the cache, verifying it on arrival, and the acceptance gate that rejects a scan whose verdict rests on layers that never ran — see **OFFLINE_RUNBOOK.md**.

Remote URLs are refused in this mode — copy the repository to the host and pass its path.

11.2 Per-tool reference



> **Offline, the staged trivy database is the ONLY dependency-CVE coverage there is.**
> Connected, four tools overlap on that job and losing one is survivable. Offline there is no redundancy.

Figure 4 — offline capability of each tool, and the one gap with no redundancy.

Group A — works offline with no preparation. Rules are compiled into the tool; nothing to stage, no flags required.

Tool	Layer	Notes
betterleaks	L1	Detection rules embedded in the binary
detect-secrets	L1	Entropy and regex rules embedded
YARA	L1	Reads your own `.yar` files from `$WORK_DIR/yara-rules/`
grep built-ins	L4	CWE-798/22/918/327/338 patterns, entirely local
PDF scanner (`scan_pdf`)	L5	Built in; needs only `python3` and its standard library — no signatures, no rules to stage, no network at any point
Makefile scanner (`scan_makefile`)	L5	Built in; pure pattern matching, nothing to stage
HTML scanner (`scan_html`)	L5	Built in; pure pattern matching, nothing to stage
Bandit	L5	Local Python AST analysis
ShellCheck	L5	Local
cppcheck	L5	Local
hadolint	L5	Local

Group B — works offline once data is staged. These are the ones that make air-gapped operation worth configuring properly.

Tool	Layer	Must be staged	Flags applied by `--offline`
Semgrep	L2	One YAML per ruleset in `$SEMGREP_RULES_DIR`	`--config <file>` instead of `p/<name>`, plus `--metrics=off`
trivy	L3	Vulnerability DB in `$TRIVY_CACHE_DIR`	`--skip-db-update --skip-java-db-update --offline-scan --cache-dir`
ClamAV	L1	`*.cvd` / `*.cld` in `$CLAMAV_DB_DIR`	`--database=<dir>`
checkov	L5	Nothing	`--skip-download` (stops remote provider-schema fetches)
ScanCode	L6	Nothing for licence/copyright	`--vulnerability` is **dropped**; licence and copyright detection are local and still run

Group C — no offline mode exists. These query a remote advisory service by design and cannot be staged. Under `--offline` they are not attempted, and each emits an `OFFLINE:` warning so the gap is on the record.

Tool	Layer	Why	Mitigation
pip-audit	L3	Resolves advisories from PyPI/OSV	Python CVEs covered by the staged trivy DB
safety	L3	Queries the Safety DB service	As above
npm audit	L3	Queries the npm registry endpoint	JS CVEs covered by the staged trivy DB
ScanCode `--vulnerability`	L6	Queries VulnerableCode	Package CVEs covered by the staged trivy DB
GitHub API size check	fetch	Needs `api.github.com`	Not applicable: `--offline` accepts local paths only

The single practical consequence: **trivy is the only dependency-CVE coverage you have offline.** If its database is not staged, Layer 3 contributes nothing.

11.2.1 Build files — `Makefile`, `.mk`, `.mak`

`scan_makefile` covers `.mk`, `.mak` and `.make` by extension, and `Makefile`, `makefile` and `GNUmakefile` by name. It is pure pattern matching, so it needs nothing staged and behaves identically offline.

It exists because a Makefile is more dangerous than it looks. Recipe lines are shell, and `$(shell …)` is evaluated when make **reads** the file — before any target runs. `make -n`, or `make` with a mistyped target name, is enough to fire it, so a hostile Makefile does not need to be built to act. Separately, Makefile assignments carry no quotes, which means `PASSWORD = hunter2` slips past the universal credential rule (that one requires a quote after the `=`).

FAIL: a download piped into a shell · a `$(shell …)` that fetches · a base64-decoded payload executed · a reverse shell · a hardcoded credential · TLS verification disabled · a setuid/setgid bit · cron or `authorized_keys` persistence · `rm -rf` of `/` or `$HOME`.

WARN: `sudo` in a recipe · `chmod 777` · a plaintext `http://` download · clone-and-build from a remote repository.

The rules are deliberately narrow, because in a Makefile the dangerous commands look almost exactly like the ordinary ones:

Ordinary — never reported	Reported
`rm -rf $(BUILD)` in a clean target	`rm -rf /`
`curl -fsSL url -o file`	`` `curl -fsSL url \ ``
`$(shell git rev-parse --short HEAD)`	`$(shell curl -s url)`
`API_KEY ?= / TOKEN = $(CI_TOKEN)`	`PASSWORD = hunter2`
`install -m 0755 prog /usr/bin`	`install -m 4755 prog /usr/bin`

Verify both directions:

```
WORK_DIR=/tmp/mkcheck bash scan_pipeline.sh tests/fixtures/rules --quiet
# Expected: evil.mk FAIL (all ten rules), Makefile FAIL (credential only),
#           safe.mk PASS with no findings at all.
```

As a false-positive check, 88 real Makefiles collected from a Debian host — dpkg, java, vim and postgresql build fragments — produce **zero findings**.

11.2.2 HTML — `.html`, `.htm`, `.xhtml`

`scan_html` is pure pattern matching, so it needs nothing staged and behaves identically offline.

Inline `<script>` is ordinary, so the rules target only constructs with no innocent explanation. Which those are was settled by counting each candidate rule against 388 real HTML files — Go tool bundles, Playwright reports, gcloud and Python package documentation — and the counts are recorded beside each rule in `scan_pipeline.sh`.

FAIL: `eval()` · a percent-encoded payload in `unescape()` · `document.write()` fed from a decoder · a script loaded over `http://` · an iframe with no visible size · `<base href="http://">` · a form posting over `http://` · a credential in a `name`/`value` attribute pair or a `<meta content>` · a cookie read wired straight to a network sink.

WARN: `innerHTML` and friends · `new Function()` · a bare `atob()` · a protocol-relative `<script src="//…">` · an external meta refresh · a `javascript:` URI.

The distinctions that keep it usable:

Ordinary — never reported	Reported
`unescape(encodeURIComponent(s))`	`unescape('%u9090%u9090')`
`<script src="https://cdn/lib.js">`	`<script src="http://cdn/lib.js">`
`<iframe src="…" width="600" height="400">`	`<iframe … style="display:none">`
`<form action="/session">`	`<form action="http://host/harvest">`
a cookie read, and a `fetch()` elsewhere	both inside one statement

```
WORK_DIR=/tmp/htmlcheck bash scan_pipeline.sh tests/fixtures/rules --quiet
# Expected: evil.html FAIL (ten rules), tracker.htm FAIL (hidden iframe),
#           safe.html PASS — it carries the near-miss of almost every rule.
```

Across those 388 real files only **two** are failed by an `scan_html` rule, both the bundled Go trace viewer, which genuinely calls `eval()`. Vendoring a file like that is what `.transit-allow.json` is for:

```
[{"rule": "CWE-95", "path": "third_party/trace_viewer_full.html",
  "reason": "vendored Go trace viewer; eval is inherent to the tool"}]
```

> **NOTE**
> .** On that same corpus the universal Layer 4 traversal rule used to fail 254 of the 388 files, matching ordinary relative links such as `href="../commands"`. It has since been narrowed to require an encoded `..`, a sensitive target, or traversal composed with input, and now flags none of them. After that change 5 of the 388 fail in total, and all 5 are true positives: three embed a private key in an example TLS config, two are the Go trace viewer's genuine `eval()`. See §11.2.4.

11.2.3 PDF scanning — no staging, and why that matters

`scan_pdf` is the one content-inspection layer with no perishable data behind it. It needs `python3` and its standard library and nothing else: no signature database, no rule bundle, no registry lookup. On an air-gapped host it is exactly as capable as on a connected one, which is not true of Layers 2, 3 or 6.

That is worth knowing because a PDF is a container, not a flat document. The format can run JavaScript the moment a file is opened (`/OpenAction`, `/AA`), start an external program (`/Launch`) and carry an embedded executable (`/EmbeddedFile`). Those are **FAIL** conditions. Things the scanner cannot see into — `/Encrypt`, `/XFA`, a stream behind a filter it cannot decode — are **WARN**, named individually, because an uninspected stream is not a clean one.

Object dictionaries need not be in plain text: from PDF 1.5 they may live inside an `/ObjStm`, so the scanner decodes each stream's declared filter chain (`FlateDecode`, `LZWDecode`, `ASCIIHexDecode`, `ASCII85Decode`, `RunLengthDecode`) before any rule runs, with a 64 MB ceiling against decompression bombs. Chains are ordinary in practice — ReportLab, which builds this very guide, writes `/Filter [ /ASCII85Decode /FlateDecode ]`.

Verify it the way the test suite does:

```
# Build the fixtures, then scan them
python3 tests/fixtures/make_pdf_fixtures.py
WORK_DIR=/tmp/pdfcheck bash scan_pipeline.sh tests/fixtures/rules --quiet

# Expected: verdict FAIL.
#   pdf_openaction_js.pdf     FAIL  CWE-94:PDF_javascript_embedded, PDF_auto_execute…
#   pdf_compressed_js.pdf     FAIL  payload was only inside a FlateDecode /ObjStm
#   pdf_obfuscated_js.pdf     FAIL  names were written /J#61vaScript
#   pdf_ascii85_js.pdf        FAIL  payload behind ASCII85Decode
#   pdf_lzw_js.pdf            FAIL  payload behind LZWDecode
#   pdf_paren_swallow_js.pdf  FAIL  stray "(" did not hide the dictionary after it
#   benign_doc.pdf            PASS  ordinary document — must not be flagged
```

The last two lines are the ones that matter on an upgrade. `benign_doc.pdf` embeds a subset font, so it carries `/AAAAAA+DejaVuSans` tags; an `/AA` rule without a right-hand anchor reads those as an AdditionalActions key and fails ordinary documentation — including this guide. And because this guide contains the words "Python/JavaScript dependency CVE", which a content stream renders as `(Python/JavaScript dependency CVE) Tj`, key rules read a view of the file with string literals blanked out. Anchoring alone cannot separate those cases: `/S/JavaScript` is valid PDF, because names self-delimit.

Acceptance test after any change to `scan_pdf` — the four PDFs this project ships are the realistic negative set:

```
mkdir -p /tmp/ownpdfs && cp *.pdf /tmp/ownpdfs/
WORK_DIR=/tmp/pdfcheck bash scan_pipeline.sh /tmp/ownpdfs --quiet
# Expected: PASS, with no findings against any of the four.
```

Not covered: an encrypted PDF is reported but not decrypted, so its streams go uninspected. Treat a `pdf:encrypted_content_could_not_be_inspected` WARN as a file needing manual review, not a clean result.

11.2.4 Traversal and key material — the rules that run on every file

Most rules are chosen by file type. Two are not: path traversal and private key material are checked on **every** file by the Layer 4 universal patterns. That reach is why both were narrowed, and why both are worth understanding before you read a report.

Path traversal. A literal `../` is a relative path, not an attack. The original rule matched any occurrence, which failed 254 of 388 real HTML files on links like `href="../commands"` and 27 of 88 real Makefiles on `-I../include` — quarantining repositories for containing documentation. Traversal is now reported only in the three contexts that indicate it:

Finding	Fires on
`CWE-22:path_traversal_encoded`	`%2e%2e, ..%2f, %252e%252e`
`CWE-22:path_traversal_to_sensitive_path`	`../../etc/passwd, ..\\windows\\system32, ../.ssh/, id_rsa`
`CWE-22:path_traversal_composed_with_input`	`"../" + name, f"{dir}/../{f}", $_GET['p'] . '../'`

So these are **not** reported, and should not be:

```
CFLAGS += -I../include                    VPATH = ../src
<a href="../commands/index.html">…</a>    from ..models import User
require('../package.json')                #include "../common/defs.h"
```

Key material. `CWE-321:hardcoded_private_key` requires a real PEM header, `-----BEGIN … PRIVATE KEY-----`. The original rule matched the bare words `PRIVATE KEY`, `CERTIFICATE` and `BEGIN RSA`, so a document that merely *discussed* keys failed the repository.

A public certificate is reported as `crypto:public_certificate_embedded`, a **WARN** rather than a FAIL: a certificate is the public half of the pair and is published by design, so it is not a leaked secret.

Verify both directions:

```
WORK_DIR=/tmp/univcheck bash scan_pipeline.sh tests/fixtures/rules --quiet
# Expected: traversal.txt and private_key.txt FAIL;
#           safe_relative_paths.txt and safe_key_prose.txt do not
#           (safe_key_prose.txt carries only the public-certificate WARN).
```

After narrowing, **5 of those 388 real HTML files fail in total and all 5 are true positives** — three embed a private key in an example TLS config, two are the bundled Go trace viewer's genuine `eval()`. The entire 88-file Makefile corpus passes.

11.3 Environment variables

Variable	Default	Purpose
`OFFLINE`	`false`	Same as `--offline`
`OFFLINE_CACHE`	`$WORK_DIR/offline-cache`	Root of the staged assets
`SEMGREP_RULES_DIR`	`$OFFLINE_CACHE/semgrep-rules`	Exported ruleset YAML files
`TRIVY_CACHE_DIR`	`$OFFLINE_CACHE/trivy-db`	trivy database cache

Variable	Default	Purpose
`CLAMAV_DB_DIR`	`$OFFLINE_CACHE/clamav`	ClamAV signature files

`--offline` also exports `SEMGREP_SEND_METRICS=off`, `DO_NOT_TRACK=1`, `CHECKPOINT_DISABLE=1` and `PIP_NO_INDEX=1`, so no tool blocks on a telemetry or version-check call.

11.4 Building the cache — on a connected host

```
./prepare_offline_cache.sh /path/to/offline-cache
```

It stages the Semgrep rulesets, the trivy database, the ClamAV signatures and your YARA rules, writes a SHA-256 manifest, and prints a summary of what it could and could not stage.

Expect roughly **300–800 MB**, dominated by the trivy database.

11.5 Transferring and verifying

```
# connected host
tar -czf offline-cache.tar.gz -C /path/to offline-cache
sha256sum offline-cache.tar.gz          # note this value

# air-gapped host, after transfer
sha256sum offline-cache.tar.gz          # must match
tar -xzf offline-cache.tar.gz -C /opt/ai-transit/
cd /opt/ai-transit/offline-cache && sha256sum --check .cache_manifest.sha256
```

11.6 Running

```
export OFFLINE_CACHE=/opt/ai-transit/offline-cache
./ai_transit.sh --offline /path/to/repo

# Docker (the wrapper forwards OFFLINE and the cache paths)
OFFLINE=true OFFLINE_CACHE=/opt/ai-transit/offline-cache \
  ./docker-run.sh --offline /path/to/repo
```

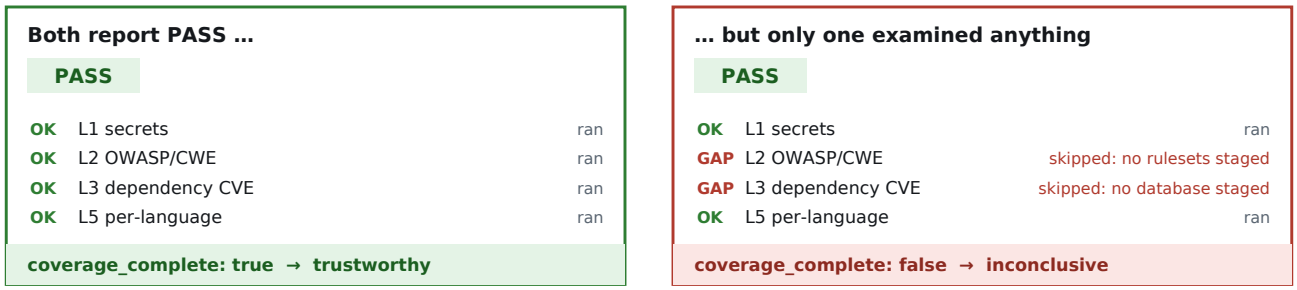
11.7 Confirming a clean offline run

The report should contain **no** `OFFLINE:` warning about Layer 2 or trivy:

```
grep -o 'OFFLINE:[^|]*' "$WORK_DIR"/reports/report_*.json
```

Warning seen	Meaning	Fix
`Layer 2 skipped entirely`	No rulesets staged — OWASP/CWE did not run	Re-run `prepare_offline_cache.sh`
`semgrep rulesets not staged (...)`	Some rulesets missing — partial coverage	Export the named rulesets
`trivy database not staged`	No dependency CVE coverage at all	Stage the trivy DB
`ClamAV signature database is empty`	Malware scan found nothing because it had no signatures	Stage `*.cvd` files
`Python/JavaScript dependency CVE scan unavailable`	Expected — see Group C	None; trivy covers this

11.8 Coverage — proving the scan actually ran



A verdict alone cannot distinguish “clean” from “nothing was examined”.
Automated consumers should gate on the coverage block, not on the verdict.

Figure 6 — two reports that both say PASS; only one is trustworthy.

Every JSON report carries a `coverage` block stating, per layer, whether it ran. This exists because a verdict alone cannot distinguish "clean" from "nothing was examined", and offline that distinction is easy to lose.

```
"coverage": {
  "L1_secrets_betterleaks": "ran",
  "L1_malware":             "skipped:no ClamAV signatures available",
  "L2_owasp_cwe":           "ran",
  "L3_dependency_cve":      "skipped:trivy database not staged for offline use",
  "L4_patterns":            "ran",
  "L5_per_language_sast":   "ran",
  "L6_licence":             "ran"
},
"coverage_complete": false,
"coverage_gaps":     ["L1_malware", "L3_dependency_cve"]
```

A layer can fail to run for two unrelated reasons — the tool is not installed, or its data was not staged — and the block records both the same way, so a consumer does not have to parse warning text to tell them apart.

Automated consumers should gate on coverage, not on the verdict:

```
REPORT=$(ls -t "$WORK_DIR"/reports/report_*.json | head -1)
python3 -c "
import json,sys
d=json.load(open(sys.argv[1]))
print('verdict :', d['verdict'])
print('gaps    :', ', '.join(d['coverage_gaps']) or 'none')
sys.exit(0 if d['coverage_complete'] else 2)
" "$REPORT"
```

`OFFLINE_RUNBOOK.md` contains a ready-made acceptance gate keyed on the five layers a verdict most depends on.

11.9 Keeping the cache current

Vulnerability data ages. A stale trivy database reports a clean result for CVEs published after it was built, which is the same failure mode as not scanning at all — only harder to notice.

Rebuild and re-transfer the cache on a defined cadence (**weekly** is a reasonable default for the trivy DB and ClamAV signatures; Semgrep rulesets change more slowly). Record the build date alongside

the archive so the air-gapped side can tell how old its data is.

12. Installing on a Disconnected Host

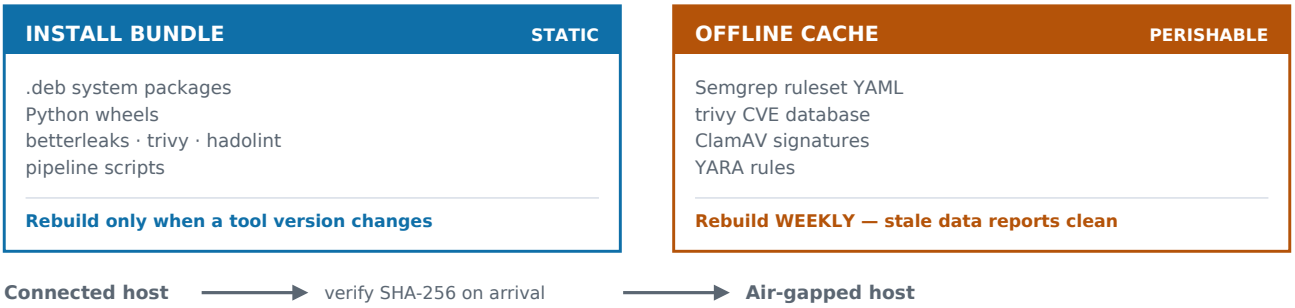


Figure 5 — the two transfer bundles and their differing lifetimes.

Section 10 covers running scans without a network. This section covers the step before it: getting the pipeline and its tools onto a machine that has never had one.

The distinction matters operationally, because the two bundles have different lifetimes:

Bundle	Contains	Rebuild when
Install bundle (this section)	The software: tools, binaries, pipeline scripts	A tool version changes
Scan cache (§11.4)	The data scanners read: rules, CVE database, signatures	**Weekly** — it is perishable

Every command in sections 3–6 assumes a network. `apt-get install` reaches archive.ubuntu.com, `pip install` reaches pypi.org, `go install` reaches the Go module proxy, and `docker build` reaches all of them plus Docker Hub. None of that works on an isolated host.

12.1 Choose a path

Path	Effort	Best when
A — Docker image transfer	Low	Docker is permitted on the target. Strongly preferred.
B — Native package staging	High	Docker is not permitted, or the target must run the tools directly.

Path A moves one file and is far less error-prone: the image already contains all sixteen tools at known versions, so nothing can be partially installed.

> **Both paths require a connected build host running the same OS release and CPU architecture as the target.** Debian packages and many Python wheels are compiled artefacts — an amd64 Ubuntu 22.04 bundle will not install on an arm64 host or on Debian 12, and the failures are confusing rather than obvious.

12.2 Path A — Docker image transfer

On the connected host:

```
./docker-run.sh --build                      # build once
./prepare_offline_install.sh /tmp/offline-install
```

That exports the image with `docker save` into the bundle. To do it by hand:

```
docker save ai-transit:latest | gzip > ai-transit.tar.gz
sha256sum ai-transit.tar.gz | tee ai-transit.tar.gz.sha256
```

Expect roughly **2–3 GB** compressed.

On the air-gapped host:

```
sha256sum -c ai-transit.tar.gz.sha256        # must pass before loading
gunzip -c ai-transit.tar.gz | docker load
docker image ls ai-transit                   # confirm it is present

# Verify it runs and is non-root
docker run --rm --entrypoint id ai-transit:latest -u    # must not be 0
```

The pipeline scripts are inside the image, but you still need them on the host to use `docker-run.sh` and to hold the scan cache. Copy `pipeline/` from the bundle to your install directory.

12.3 Path B — Native package staging

On the connected host (same OS and architecture as the target):

```
./prepare_offline_install.sh /tmp/offline-install
```

It stages four groups:

Directory	Contents	How it was obtained
`deb/`	System packages and their dependencies	`apt-get install --download-only`
`wheels/`	Python packages	`pip3 download`
`bin/`	betterleaks, trivy, hadolint	Copied from the build host's PATH
`pipeline/`	Scripts, tests, documentation	Copied from the repository

`bin/` is populated from the tools already installed on the build host, so install them there first (§6) before running the script.

Transfer and verify:

```
tar -czf offline-install.tar.gz -C /tmp offline-install
sha256sum offline-install.tar.gz | tee offline-install.tar.gz.sha256

# On the air-gapped host
sha256sum -c offline-install.tar.gz.sha256
tar -xzf offline-install.tar.gz -C /opt/
cd /opt/offline-install
sha256sum --check .install_manifest.sha256 | grep -v ': OK$' || echo "all files OK"
```

Check the platform matches before installing anything:

```
cat .bundle_platform
# os_id=ubuntu  os_version=22.04  arch=amd64  built_on=...

. /etc/os-release; echo "target: ${ID} ${VERSION_ID} $(dpkg --print-architecture)"
# These must agree. If they do not, rebuild the bundle on a matching host.
```

Install:

```
# 1. System packages — dpkg resolves nothing, so install the whole set at once
sudo dpkg -i deb/*.deb || sudo apt-get install -f --no-download -y

# 2. Python packages — --no-index forbids any network fallback
python3 -m venv /opt/ai-transit/venv
source /opt/ai-transit/venv/bin/activate
pip install --no-index --find-links=wheels \
    openpyxl reportlab python-docx detect-secrets bandit \
    pip-audit safety semgrep checkov scancode-toolkit

# 3. Binaries
sudo cp bin/* /usr/local/bin/ && sudo chmod +x /usr/local/bin/{betterleaks,trivy,hadolint}

# 4. Pipeline
sudo mkdir -p /opt/ai-transit
sudo cp -r pipeline/* /opt/ai-transit/
sudo chmod +x /opt/ai-transit/*.sh
```

`pip install --no-index` is deliberate: without it, pip silently falls back to PyPI and the install appears to succeed on a host that merely has *partial* network access, hiding the fact that the bundle was incomplete.

12.4 Verify the installation

```
cd /opt/ai-transit

# 1. Which tools actually made it
for t in betterleaks detect-secrets clamscan yara semgrep trivy \
         bandit shellcheck cppcheck hadolint checkov scancode; do
    command -v "$t" >/dev/null && echo "  present : $t" || echo "  MISSING : $t"
done

# 2. The suite needs no tools and no network — it should pass regardless
./tests/run_tests.sh
# Expected: ✔ 108/108 passed

# 3. Record the bundle for the integrity check
python3 selfcheck.py --write-manifest
```

Anything reported `MISSING` will WARN on every scan and never contribute findings. Resolve it now rather than discovering it in a report later — §11.8 explains how the coverage block makes such gaps visible.

12.5 Then stage the scan data

Installing the tools is only half the job. The scanners still need their rules and databases, which is a **separate and perishable** bundle:

```
# On the connected host
./prepare_offline_cache.sh /tmp/offline-cache
```

Follow §11.4–10.6, or the step-by-step procedure in **OFFLINE_RUNBOOK.md**.

13. Sample Scans — Testing the Pipeline

13.1 Quick smoke test (local directory)

Create a small test repository to validate the pipeline end-to-end:

```
#!/usr/bin/env bash
# create_test_repo.sh — creates a clean sample repo that should PASS

mkdir -p /tmp/sample-repo/{src,tests,infra}

# Python source
cat > /tmp/sample-repo/src/app.py << 'EOF'
import os
import sqlite3

def get_user(db_path: str, username: str) -> list:
    """Retrieve user using parameterised query (safe)."""
    conn = sqlite3.connect(db_path)
    return conn.execute(
        "SELECT * FROM users WHERE name = ?", (username,)
    ).fetchall()

def main():
    db = os.environ.get("DB_PATH", "/tmp/app.db")
    print(get_user(db, "alice"))
EOF

# Shell script
cat > /tmp/sample-repo/src/setup.sh << 'EOF'
#!/usr/bin/env bash
set -euo pipefail
APP_DIR="${APP_DIR:-/opt/app}"
mkdir -p "$APP_DIR"
echo "Setup complete"
EOF

# Requirements file (up-to-date)
cat > /tmp/sample-repo/requirements.txt << 'EOF'
requests==2.32.3
Pillow==10.3.0
EOF

# README
echo "# Sample project" > /tmp/sample-repo/README.md

echo "Test repo created at /tmp/sample-repo"
```

```
bash create_test_repo.sh
./ai_transit.sh /tmp/sample-repo
# Expected: PASS — ZIP produced in ./Good/
```

13.2 FAIL test — embedded secret

```
mkdir -p /tmp/fail-secret

cat > /tmp/fail-secret/config.py << 'EOF'
# Bad practice: hardcoded credential
AWS_ACCESS_KEY_ID = "AKIAIOSFODNN7EXAMPLE"
AWS_SECRET_ACCESS_KEY = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
EOF

./ai_transit.sh /tmp/fail-secret
# Expected: FAIL — betterleaks or detect-secrets detects the AWS keys
# Files moved to /opt/ai-transit/quarantine/

rm -rf /tmp/fail-secret
```

13.3 FAIL test — SQL injection (OWASP A03)

```
mkdir -p /tmp/fail-sqli

cat > /tmp/fail-sqli/db.py << 'EOF'
import sqlite3

def login(username, password):
    conn = sqlite3.connect("app.db")
    query = "SELECT * FROM users WHERE name='" + username + \
            "' AND pass='" + password + "'"
    return conn.execute(query).fetchall()
EOF

./ai_transit.sh /tmp/fail-sqli
# Expected: FAIL — Semgrep detects SQL injection (OWASP A03 / CWE-89)

rm -rf /tmp/fail-sqli
```

13.4 FAIL test — vulnerable dependency (CVE)

```
mkdir -p /tmp/fail-cve

cat > /tmp/fail-cve/requirements.txt << 'EOF'
Pillow==9.0.0
requests==2.18.0
EOF

./ai_transit.sh /tmp/fail-cve
# Expected: FAIL — trivy/pip-audit detect CVEs in Pillow 9.0.0

rm -rf /tmp/fail-cve
```

13.5 WARN test — risky licence (GPL)

```
mkdir -p /tmp/warn-licence

cat > /tmp/warn-licence/main.py << 'EOF'
# This file is licensed under the GNU General Public License v3.0
```

```
# Copyright (C) 2024 Example Corp.
# You may redistribute and/or modify under the terms of the GPL.

def hello():
    print("hello")
EOF

./ai_transit.sh /tmp/warn-licence
# Expected: PASS (WARN logged) — ScanCode detects GPL-3.0 licence
# ZIP produced in ./Good/ but WARN logged for legal review

rm -rf /tmp/warn-licence
```

13.6 FAIL test — Dockerfile misconfigurations

```
mkdir -p /tmp/fail-docker

cat > /tmp/fail-docker/Dockerfile << 'EOF'
FROM ubuntu:latest
ENV DB_PASSWORD=mysecretpassword
RUN apt-get update && apt-get install -y curl
ADD https://example.com/app.tar.gz /app/
EOF

./ai_transit.sh /tmp/fail-docker
# Expected: FAIL — hadolint detects :latest tag, ENV secret, ADD from URL

rm -rf /tmp/fail-docker
```

13.7 Scan a real public GitHub repository

```
# Scan a small, well-known public repo (adjust URL as needed)
./ai_transit.sh https://github.com/psf/requests main
# Expected: PASS or WARN — clean repo, dependencies may have minor CVEs
```

13.8 Run the self-check to verify the pipeline itself

```
# PDF report (default)
python3 selfcheck.py --bundle-dir . --output selfcheck_report.pdf

# JSON report (machine-readable, CI-friendly)
python3 selfcheck.py --bundle-dir . --format json --output selfcheck_report

# Both formats at once
python3 selfcheck.py --bundle-dir . --format both --output selfcheck_report

# Run only specific checks (faster)
python3 selfcheck.py --bundle-dir . --only 11.1,11.4,11.6

# Expected output:
#   11.1 Meta-scan        → PASS or WARN
#   11.2 Binary checks    → SKIP (unless --checksums provided)
#   11.3 GPG/cosign       → PASS or WARN
#   11.4 Python CVE       → PASS
```

```
#   11.5 Host OS CVE      → PASS or WARN
#   11.6 Bundle integrity → PASS
#   11.7 AIDE             → SKIP (unless AIDE installed)
```

13.9 CI mode — quiet verdict with severity filter

```
# Only block on CRITICAL CVEs/findings; warnings and lower are ignored
./ai_transit.sh --quiet --min-severity critical https://github.com/org/repo
echo "Exit code: $?"   # 0 = PASS, 1 = FAIL
```

13.10 Diff mode — scan only changed files (PR workflow)

```
# Scan only files changed since the merge base commit
SINCE=$(git merge-base HEAD origin/main)
./ai_transit.sh --since "$SINCE" /path/to/local/repo
```

13.11 Private repository scan

```
# Generate a fine-grained PAT with "Contents: read" on github.com
export GITHUB_TOKEN="ghp_your_token_here"
./ai_transit.sh https://github.com/myorg/private-repo main
```

13.12 Report-only mode (audit without blocking)

```
# Scan and generate reports but always exit 0 — useful for first-pass auditing
./ai_transit.sh --report-only https://github.com/org/repo
# Reports are written to $WORK_DIR/reports/ regardless of findings
```

13.13 Air-gapped scan

```
# On a connected host: build the cache and transfer it
./prepare_offline_cache.sh /tmp/offline-cache
tar -czf offline-cache.tar.gz -C /tmp offline-cache

# On the air-gapped host
tar -xzf offline-cache.tar.gz -C /opt/ai-transit/
cd /opt/ai-transit/offline-cache && sha256sum --check .cache_manifest.sha256

export OFFLINE_CACHE=/opt/ai-transit/offline-cache
./ai_transit.sh --offline /path/to/repo

# Confirm no layer was silently skipped
grep -o 'OFFLINE:[^|]*' "$WORK_DIR"/reports/report_*.json
# Expected: only the Python/JavaScript dependency notices (see 10.2 group C)
```

14. Self-Scan: Verifying the Installation is Safe

Before deploying in a production environment, run the full self-check:

```
python3 selfcheck.py --bundle-dir . --output selfcheck_report.pdf
```

This executes all self-check checks (meta-scan, binary checksums, GPG/cosign, Python CVE scan, host OS CVE, bundle integrity, AIDE) and produces a colour-coded PDF report. See `selfcheck.py --help` for all options.

14.1 Output formats and selective checks

```
python3 selfcheck.py --format json --output selfcheck_report   # machine-readable
python3 selfcheck.py --format both --output selfcheck_report   # PDF + JSON
python3 selfcheck.py --only 11.1,11.4,11.6                     # subset, faster
```

The JSON report carries a top-level `verdict` field (`PASS` / `WARN` / `FAIL`), which is what a CI job should key on.

14.2 The bundle manifest — generate it at install time

Self-check 11.6 (bundle file integrity) compares every bundle file against `.bundle_manifest.sha256`. That manifest is **deliberately not tracked in version control**: if it were, it would report "File tampering detected" after every ordinary edit, and a check that cries wolf is a check people learn to ignore. Git already guarantees the integrity of the repository; the manifest's job is to detect tampering with an **installed** bundle.

Generate it once, immediately after installing and verifying the bundle:

```
python3 selfcheck.py --write-manifest
```

Regenerate it after any *intentional* change to the bundle. From then on, any self-check 11.6 failure means a file changed without your knowledge.

```
# Verify (should PASS on an untouched installation)
python3 selfcheck.py --only 11.6
```

The manifest stores **relative** paths, so it stays valid if the bundle is moved or installed to a different prefix. Verification must therefore be run from the bundle directory (selfcheck.py does this automatically).

15. Security Hardening Recommendations

Dedicated service account

```
sudo useradd -r -m -d /opt/ai-transit -s /bin/bash aitransit
sudo chown -R aitransit:aitransit /opt/ai-transit
sudo -u aitransit ./ai_transit.sh https://github.com/org/repo
```

Network isolation (allow only github.com outbound)

```
sudo ufw default deny outgoing
sudo ufw allow out 443 comment "HTTPS for GitHub"
sudo ufw allow out 53  comment "DNS"
sudo ufw enable
```

Quarantine filesystem (noexec)

```
# /etc/fstab
```

```
tmpfs /opt/ai-transit/quarantine tmpfs rw,noexec,nosuid,nodev,size=2G 0 0
```

Weekly tool updates (cron)

```
# /etc/cron.weekly/ai-transit-update
#!/bin/bash
freshclam
trivy image --download-db-only
source /opt/ai-transit/venv/bin/activate
pip install --upgrade betterleaks detect-secrets semgrep bandit \
    pip-audit safety checkov scancode-toolkit
```

Log rotation

```
# /etc/logrotate.d/ai-transit
/opt/ai-transit/logs/*.log {
    daily
    rotate 30
    compress
    missingok
    notifempty
}
```

16. Troubleshooting

Symptom	Likely cause	Fix
`betterleaks: command not found`	Binary not in PATH	`sudo cp ~/go/bin/betterleaks /usr/local/bin/`
`freshclam: connect refused`	clamav-freshclam not running	`sudo systemctl start clamav-freshclam`
`semgrep: network error`	No internet — first run	Pre-download rules (see §11)
`trivy: DB not found`	First run without internet	`trivy image --download-db-only`
`scancode: takes too long`	Large repo	Use `--timeout 30` to limit per-file time
`pip-audit: no vulnerabilities` on old packages	DB unreachable	Check internet; use trivy offline instead
`npm audit: ENOLOCK`	No package-lock.json	Run `npm install --package-lock-only` first
`openpyxl manquant`	Not installed in venv	`pip install openpyxl`
`declare -A: invalid option`	Bash < 4.0	Install bash 5: `brew install bash` / use WSL2
`zip: command not found`	zip not installed	`sudo apt-get install zip`
WARN on all files	Missing optional tools	Install missing tools; WARNs do not block
Git clone fails on a private repo	No credentials	Set `GITHUB_TOKEN` (see §9.2); the token is passed via `GIT_ASKPASS`, never in the clone URL

Symptom	Likely cause	Fix
`Repository not found or private (HTTP 404)`	Private repo without a token, or a typo	Set `GITHUB_TOKEN`, or check the URL
self-check 11.6 reports "File tampering detected" after your own edits	Manifest is stale	Regenerate it: `python3 selfcheck.py --write-manifest` (see §13.2)
`--only` reports 0 checks and exits 2	Check IDs mistyped (`1.1` instead of `11.1`)	Use the full IDs: `11.1` … `11.7`
Findings appear against the wrong file	Report written by a pre-P8 version	Upgrade; the JSON writer dropped empty records and shifted rows
Allowlist entries have no effect	`.transit-allow.json` not at the repository root, or `rule`/`path` do not match	`path` is relative to the repo root; `rule` matches the finding's leading token, e.g. `CWE-89`
Diff mode scans everything	`--since` commit unreachable in a shallow clone	The pipeline warns and falls back to a full scan; fetch more history or use a local path
Docker build fails downloading trivy	Pinned version no longer published	Update `ARG TRIVY_VERSION` / `TRIVY_SHA256` (see §19.1); the `pins` CI job prints the correct digest
ZIP contains `tmp/…/fetch/repo_…` paths	Archive built by a pre-P8 version	Upgrade; archives are now rooted at the repository
Scan hangs for minutes on an isolated host	Running without `--offline`; tools are blocking on network timeouts	Use `--offline` (see §11)
Offline run passes suspiciously fast with few findings	Layers 2 and 3 had no staged data	Check for `OFFLINE:` warnings in the report; stage the cache (§11.4)
`OFFLINE:Layer 2 skipped entirely`	Semgrep rulesets not staged	Run `prepare_offline_cache.sh` on a connected host
`OFFLINE:trivy database not staged`	No dependency CVE coverage offline	Stage the trivy DB; it is the only offline CVE source
`Cannot clone … without a network`	`--offline` with a remote URL	Copy the repository to the host and pass its path

17. Running the Test Suite

The suite verifies the pipeline itself: that rules fire on unsafe code, that they **do not** fire on safe code, that flags behave, and that the reports and archive are well-formed.

```
./tests/run_tests.sh          # everything
./tests/run_tests.sh -v       # show detail for failures
./tests/run_tests.sh rules    # only groups whose name matches "rules"
```

It requires **no scanning tools at all**. With none installed the pipeline degrades to its built-in grep rules and every assertion still holds — that is exactly how CI runs it. Tools that happen to be present are used, but no assertion depends on them.

Expected output on a healthy installation:

```
── Layer A — rule corpus (detection correctness)
```

```
  ✔ sql_injection.py is flagged for SQL injection
  ✔ does NOT flag parameterised SQL (execute("… = ?", (v,)))
  …
──────────────────────────────────────────

  ✔  108/108 passed
```

17.1 What each layer covers

Layer	Covers
A — rule corpus	Detection correctness: each finding must land on the correct file, plus false-positive guards for safe code
B — end-to-end	Clean repo → PASS/exit 0; vulnerable repo → FAIL/exit 1
C — flags	`--report-only`, `--min-severity`, argument guards, `.transit-allow.json`, `.transitignore`, `--no-zip`/`--no-excel`
D — artifacts	JSON valid and carries a verdict, HTML written, ZIP entries repo-relative, Excel Findings tab, no escape codes when redirected
E — diff mode	`--since` scans exactly the changed files; `.git` excluded from local copies
F — static	Parse checks, shellcheck, and lint rules for two bug classes that have already shipped

17.2 Fixtures

`tests/fixtures/` holds small repositories, each with one job:

Fixture	Expected
`clean/`	PASS
`vulnerable/`	FAIL
`allowlisted/`	PASS — the finding is downgraded by `.transit-allow.json`
`ignored/`	PASS — the offending file is excluded by `.transitignore`
`rules/`	A corpus where each file triggers, or deliberately does not trigger, one rule

`rules/safe_sql.py` is a regression guard: it contains correct parameterised queries that a previous version of the SQL rule wrongly flagged as injection.

17.3 Adding a rule

Add **both** a file that must trigger the rule and a similar-but-safe file that must not, then confirm the new assertion **fails before the rule exists**. A test that has never been observed failing proves nothing — during development of this suite, two tests passed against known-broken code because the tests themselves were wrong.

18. Continuous Integration

`.github/workflows/ci.yml` runs on every push and pull request.

Job	Purpose	Blocking
`lint`	shellcheck (errors fatal, warnings advisory) + Python syntax	Yes
`test`	Test suite with no scanning tools — the fast signal	Yes
`test-with-tools`	Suite again with semgrep/bandit/detect-secrets installed; exercises tool-dependent paths the degraded run cannot reach	No
`pins`	Downloads each pinned tool artifact, fails if the version does not exist, and compares against the pinned SHA-256	Yes
`docker`	Builds the image, asserts it runs as non-root, smoke-tests both fixtures	Yes

The `docker` job is the only place the multi-stage build, the pinned tool versions and the non-root user are actually exercised — none of that can be verified by the test suite alone.

If `pins` fails, the pinned version in the `Dockerfile` no longer resolves to a downloadable artifact. The job log prints the correct SHA-256 for the current pin; update `ARG TRIVY_VERSION` / `ARG TRIVY_SHA256` accordingly.

19. Docker Image — Build Arguments & Integrity

The image is a two-stage build: a `builder` stage compiles betterleaks with Go, and the runtime stage copies only the resulting binary, so the Go toolchain never reaches the final image. It runs as the non-root user `transit`.

19.1 Build arguments

Argument	Default	Purpose
`TRIVY_VERSION`	`0.58.2`	trivy release to install
`TRIVY_SHA256`	*(empty)*	SHA-256 of the trivy tarball; empty disables verification
`HADOLINT_VERSION`	`2.12.0`	hadolint release to install
`HADOLINT_SHA256`	`56de6d5e…`	SHA-256 of the hadolint binary (verified)
`BETTERLEAKS_VERSION`	`0.1.0`	betterleaks module version

```
docker build -t ai-transit:latest \
  --build-arg TRIVY_VERSION=0.58.2 \
  --build-arg TRIVY_SHA256=<digest> .
```

19.2 Why the digests matter

Pinning a version defends against getting a *different release*. It does not defend against getting a *different binary* for that release — a compromised or replaced artifact keeps the same version string. The digest closes that gap.

When a digest argument is empty the build still succeeds, but prints a warning **and the artifact's actual digest**, so the correct value can be pasted in without a separate download. The `pins` CI job prints the same value.

> `TRIVY_SHA256` ships empty because the pinned trivy version could not be resolved from the development environment. Fill it in from the first `pins` CI run before treating the image as production-ready.

19.3 Running through the wrapper

`docker-run.sh` forwards pipeline flags into the container, so the Docker and native interfaces behave identically:

```
./docker-run.sh --build                                          # build once
./docker-run.sh --quiet --min-severity critical https://github.com/org/repo
./docker-run.sh --report-only /path/to/local/repo
```

Environment variables (`WORK_DIR`, `MAX_SIZE_MB`, `MIN_SEVERITY`, `VERBOSITY`, `GITHUB_TOKEN`, `SINCE_COMMIT`) are forwarded automatically when set.

> A `GITHUB_TOKEN` passed to a container is visible via `docker inspect` to anyone who can reach the Docker daemon. On a shared host, prefer running the pipeline natively for private repositories.

Quick Reference — One-Line Install (Ubuntu 22.04 / 24.04)

```
# Step 1: system packages
sudo apt-get update && sudo apt-get install -y \
    bash git curl jq zip unzip file coreutils wget gpg \
    python3 python3-pip python3-venv build-essential golang-go \
    nodejs npm shellcheck cppcheck clamav clamav-daemon yara

# Step 2: Python venv + packages
python3 -m venv /opt/ai-transit/venv
source /opt/ai-transit/venv/bin/activate
pip install --upgrade pip
pip install openpyxl reportlab detect-secrets bandit pip-audit safety \
    semgrep checkov scancode-toolkit

# Step 3: betterleaks
go install github.com/betterleaks/betterleaks@latest
sudo cp ~/go/bin/betterleaks /usr/local/bin/betterleaks

# Step 4: trivy
curl -sfL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh \
    | sudo sh -s -- -b /usr/local/bin

# Step 5: hadolint
VERSION=$(curl -s https://api.github.com/repos/hadolint/hadolint/releases/latest \
    | jq -r '.tag_name')
sudo curl -sSL \
    "https://github.com/hadolint/hadolint/releases/download/${VERSION}/hadolint-Linux-x86_64" \
    -o /usr/local/bin/hadolint && sudo chmod +x /usr/local/bin/hadolint

# Step 6: ClamAV DB update
sudo freshclam
```

```
# Step 7: trivy DB
trivy image --download-db-only

echo "Installation complete — run ./verify_install.sh to confirm"
```

AI Transit Pipeline — Installation Guide v2.1 — 2025